

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

PRACTICAL - 8

AIM :- To implement a passive reinforcement learning algorithm based on Adaptive Dynamic Programming (ADP) for a smart classroom temperature-control problem.

Code:-

```
#T101 RAJDEEP M PARAB
# AIM: Implement Passive Reinforcement Learning based on Adaptive Dynamic Programming (ADP) for a
Smart Room Cleaning Problem.
import numpy as np
# Function to calculate the utility of a state using the Bellman equation.
def return_state_utility(v, T, u, reward, gamma):
    # Four possible actions:
    # 0 = Up, 1 = Left, 2 = Right, 3 = Down
    action_array = np.zeros(4)
    for action in range(0, 4):
        # Calculate the expected utility for the action
        action_array[action] = np.sum(
            np.multiply(u, np.dot(v, T[:, :, action])))
    )
    # Bellman equation
    return reward + gamma * np.max(action_array)

# Function to learn the transition model
def learn_transition_model(episodes, num_states, num_actions):
    # Count how many times each transition occurs
    transition_counts = np.zeros(
        (num_states, num_states, num_actions)
    )
    # Count how many times each action is performed
    action_counts = np.zeros(
        (num_states, num_actions)
    )
    # Learn from every episode
    for episode in episodes:
        for state, action, next_state in episode:
            transition_counts[state, next_state, action] += 1
            action_counts[state, action] += 1
    # Convert counts into probabilities
    T = np.zeros(
        (num_states, num_states, num_actions)
    )
    for state in range(num_states):
```

T101

RAJDEEPP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

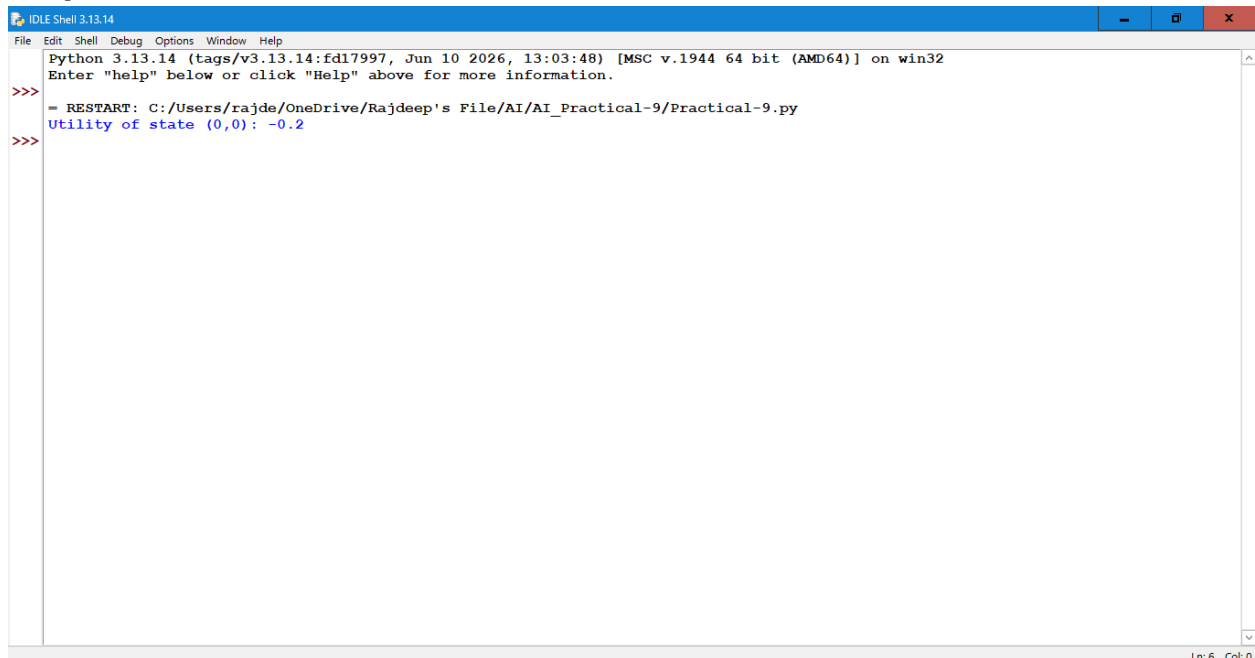
```
for action in range(num_actions):
    total = action_counts[state, action]
    if total > 0:
        T[:, state, action] = (
            transition_counts[state, :, action] / total
        )
    return T
def main():
    # 3 x 4 Smart Room
    # Grid:
    # (0,0) (0,1) (0,2) (0,3)
    # (1,0) (1,1) (1,2) (1,3)
    # (2,0) (2,1) (2,2) (2,3)
    # State 5 = Obstacle
    # State 3 = Charging Station
    num_states = 12
    num_actions = 4
    # Starting state is (0,0)
    v = np.zeros((1, num_states))
    v[0, 0] = 1.0
    # Sample experience collected by the agent.
    # Each tuple contains:(current_state, action, next_state)
    # Actions:
    # 0 = Up
    # 1 = Left
    # 2 = Right
    # 3 = Down
    episodes = [
        # Episode 1
        [(0, 2, 1), (1, 2, 2), (2, 2, 3)],
        # Episode 2
        [(0, 2, 1), (1, 2, 2), (2, 2, 3)],
        # Episode 3
        [(0, 2, 1), (1, 2, 2), (2, 2, 3)],
        # Episode 4
        [(0, 2, 1), (1, 2, 2), (2, 2, 3)],
        # Episode 5
        [(0, 2, 1), (1, 2, 2), (2, 2, 3)]
    ]
    # Learn the transition model using ADP.
    T = learn_transition_model(
        episodes,
        num_states,
        num_actions
    )
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
# Utility vector.
# State 3 is the charging station.
# State 5 is an obstacle.
u = np.array([[
    0.512, 0.640, 0.800, 1.000,
    0.420, 0.000, 0.350, 0.280,
    0.300, 0.250, 0.200, 0.150
]])
# Reward for the starting state
reward = -0.2
# Discount factor
gamma = 1.0
# Calculate utility of starting state (0,0)
utility_00 = return_state_utility(
    v, T, u, reward, gamma
)
print("Utility of state (0,0):", round(utility_00, 4))
main()
```

Output:



```
IDLE Shell 3.13.14
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI_Practical-9/Practical-9.py
Utility of state (0,0): -0.2
>>>
```